SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Dry Run Evaluation

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO1 – Apply (BL3)	**Assessment:**	Assessment Tool 2 – Dry Run Evaluation
Weightage:	35% of CIA	**Duration / Marks:**	60 mins / 100 marks
CO1 Statement:	Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)		

Student Name:	B.Muni Chandini	**Reg. No.:**	192524283
Section / Batch:	AI&DS	**Date:**	10/10/2026

Code of Conduct

I B.Muni Chandini (192524283) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:*B.Muni Chandini*

Instructions

• Read each question carefully; answer all four questions (25 marks each, 100 in total).

• Show every dry-run step: the initial queue, stack, priority queue or belief state, each iteration, your assumptions and a clear final answer, and finish with the interpretation asked for.

• Follow the conventions stated in each question (alphabetical expansion, explored set, goal test on removal from the frontier, tie-breaking) and present tables neatly.

• Marks follow the Dry Run Evaluation rubric: understanding of the search problem, state expansion and dry run execution, application of the search algorithm, accuracy of the solution path, and presentation and logical reasoning.

• Upload in GitHub and GCR (question paper with Reg. No., Name, answers, rubrics and evaluation page at the end). Duration: 1 hour (15 minutes per question).

Section / Topic	Focus	Question Nos.
1. Foundations and History of AI	Four approaches to AI, Turing Test, eras and milestones	Q2
2. Intelligent Agents: Agents and Environments	Agent, percepts, actions, agent function and program, PEAS	Q1
3. The Concept of Rationality	Performance measure, percept sequence, rational vs omniscient	Q1

4. The Nature of Environments	Task-environment properties and their classification	Q1
5. The Structure of Agents	Simple reflex and model-based reflex agent programs	Q1
6. Problem-Solving Agents	Goal formulation, problem formulation, search, execution	Q2
7. Problem Formulation	States, initial state, actions, goal test, path cost; belief states	Q2, Q4
8. Searching for Solutions	Search graph, frontier, explored set, node expansion	Q3
9. Uninformed Search Strategies	Dry runs of BFS, DFS and uniform-cost search	Q3
10. Search with Partial Information	Sensorless problems, belief states, percept update	Q4

Annexure A – Dry Run Evaluation

Question 1: Agents, Environments and Rationality – Dry Run of Agent Programs

Topic: Intelligent agents, rationality, nature of environments, structure of agents |

A vacuum-cleaner agent lives in a world with two squares, A and B. At each time step it perceives [its location, status of its own square] and performs one action: Suck, Right, Left or NoOp. The agent starts in square A, both squares are initially Dirty, and dirt never reappears.

Performance measure over T = 6 time steps: +10 for every square cleaned (a Suck on a Dirty square) and −1 for every Left or Right move.

Agent X (simple reflex): if the current square is Dirty, Suck; otherwise move to the other square.

Agent Y (model-based reflex): keeps an internal state of the squares it knows to be clean. If the current square is Dirty, Suck; otherwise, if both squares are known clean, NoOp; otherwise move to the other square.

(a) PEAS description

Component	Description
Performance	+10 for every square cleaned (Suck on Dirty); −1 for every Left or Right move. Evaluated over T = 6 steps.
Environment	Two-square vacuum world (squares A and B). Each square is Clean or Dirty. Dirt never reappears. Agent starts in A; both initially Dirty.
Actuators	Suck, Right, Left, NoOp
Sensors	Location sensor and dirt sensor for the current square → percept = [location, status]

(b) Task-environment classification

Dimension	Classification	Justification
Observability	Partially observable	Agent senses only its current square; the other square's status is not directly observed.
Number of agents	Single-agent	Only one vacuum-cleaner agent acts in the environment.
Determinism	Deterministic	Actions have predictable, fixed outcomes (Suck cleans, move changes location).
Episodic / Sequential	Sequential	Current decisions affect future states and future performance (cleaning persists).
Static / Dynamic	Static	Environment does not change while the agent is deliberating; dirt never reappears.
Discrete / Continuous	Discrete	Finite discrete locations, dirt statuses, and action set.

(c) Dry-run of both agents for 6 time steps

Assumptions: Agent starts in A; both A and B are Dirty. 'Move to the other square' means Right from A, Left from B. Y's known-clean set is updated after a successful Suck.

Step	X: Percept	X: Action	Y: Percept	Y: Action	Y: Known-Clean Squares
1	[A, Dirty]	Suck	[A, Dirty]	Suck	{A}
2	[A, Clean]	Right	[A, Clean]	Right	{A}
3	[B, Dirty]	Suck	[B, Dirty]	Suck	{A, B}
4	[B, Clean]	Left	[B, Clean]	NoOp	{A, B}
5	[A, Clean]	Right	[B, Clean]	NoOp	{A, B}
6	[B, Clean]	Left	[B, Clean]	NoOp	{A, B}

(d) Total performance of X and Y

Agent X: Cleaned 2 squares (+20). Moves: Right (step 2), Left (step 4), Right (step 5), Left (step 6) → 4 moves (−4). Total = 20 − 4 = +16.

Agent Y: Cleaned 2 squares (+20). Moves: Right (step 2) only → 1 move (−1). NoOp on steps 4–6. Total = 20 − 1 = +19.

(e) Interpretation – Rationality

Which agent is rational? Agent Y is rational with respect to the given performance measure. It maximises expected performance by stopping once both squares are known clean, thereby avoiding unnecessary move penalties. Agent X keeps oscillating and incurs extra −1 costs, so it is not rational under this measure.

Why different actions on identical percept [B, Clean] at step 4?

• Agent X is a simple-reflex agent: its program maps the current percept alone to an action. Rule: 'if Clean then move to the other square'. Hence it executes Left.

• Agent Y is a model-based reflex agent: it maintains an internal model (known-clean set). At step 4 the set is already {A, B}, so the rule 'if both known clean → NoOp' fires.

Rational vs omniscient: A rational agent selects the action that maximises expected performance given the percept sequence so far and its knowledge. An omniscient agent would know the actual outcomes of its actions and the true state of the whole world in advance; rationality does not require omniscience—only the best decision with the information available.

Question 2: Foundations and History of AI; Problem-Solving Agent and Problem Formulation

Topic: Foundations and history of AI, problem-solving agents, problem formulation |

Part A (a) Classification table

System / Idea	Approach to AI	Era / Milestone
Turing Test proposed (1950)	Acting humanly	Birth of AI / Dartmouth era precursor (1950)
General Problem Solver (Newell and Simon, 1957)	Thinking humanly	Early AI / Cognitive simulation (1950s–60s)
Aristotle's syllogisms and formal logic	Thinking rationally	Foundations / Ancient logic (pre-AI roots)
An agent that does the right thing to maximise expected performance	Acting rationally	Modern / Rational-agent approach (post-1980s)

Part B – Water-Jug Problem

Two jugs: X (4 L) and Y (3 L), both initially empty. Goal: exactly 2 litres in jug X.

(b) Problem formulation

• State representation: (x, y) where x ∈ {0,1,2,3,4} is the current volume in jug X and y ∈ {0,1,2,3} is the current volume in jug Y.

• Initial state: (0, 0)

• Actions: Fill(X), Fill(Y), Empty(X), Empty(Y), Pour(X→Y), Pour(Y→X)

• Transition model:

  – Fill(X): (4, y) Fill(Y): (x, 3)

  – Empty(X): (0, y) Empty(Y): (x, 0)

  – Pour(X→Y): let t = min(x, 3−y); result (x−t, y+t)

  – Pour(Y→X): let t = min(y, 4−x); result (x+t, y−t)

• Goal test: x = 2 (any y is acceptable)

• Path cost: 1 per action (unit cost)

(c) Successor states

Successors of (0, 0) (unchanged actions omitted):

Fill(X) → (4, 0) Fill(Y) → (0, 3)

Successors of (4, 0):

Empty(X) → (0, 0) Fill(Y) → (4, 3) Pour(X→Y) → (1, 3)

(d) State-space size, shortest solution, path cost

Number of states: 5 × 4 = 20 possible states (all combinations of x = 0..4, y = 0..3).

A shortest solution (6 actions, path cost = 6):

(0,0) → Fill(X) → (4,0) → Pour(X→Y) → (1,3) → Empty(Y) → (1,0) → Pour(X→Y) → (0,1) → Fill(X) → (4,1) → Pour(X→Y) → (2,3)

Goal reached: x = 2. Path cost = 6.

(e) Four phases of a problem-solving agent

1. Goal formulation: Decide that the goal is ‘exactly 2 litres in jug X’.

2. Problem formulation: Abstract the world into states (x,y), actions, transition model, goal test and path cost as above.

3. Search: Explore the state space (e.g., BFS) offline to find a sequence of actions that reaches a goal state.

4. Execution: Carry out the action sequence found by search.

Why no new percepts are needed: The environment is fully observable, deterministic and static. Once a solution path has been computed, every intermediate state is known in advance; the agent can therefore execute the open-loop plan without sensing.

Question 3: Searching for Solutions – Uninformed Search Strategies

Topic: Searching for solutions, BFS, DFS, uniform-cost search |

Graph (undirected, weighted). Start = S, Goal = G. Graph search with explored set; expand neighbours alphabetically; goal test on removal from frontier; ties broken alphabetically.

Edge	Cost	Edge	Cost
S – A	1	B – D	1
S – B	4	C – G	6
A – C	2	D – G	2
A – D	5		

Alphabetical neighbours: S→{A,B}, A→{C,D,S}, B→{D,S}, C→{A,G}, D→{A,B,G}, G→{C,D}.

(a) BFS from S

Queue initially [S]. Do not add a node already explored or already in the queue. Goal test when node is dequeued.

Iter	Node Dequeued	Queue after Expansion	Explored Set
1	S	[A, B]	{S}
2	A	[B, C, D]	{S, A}
3	B	[C, D]	{S, A, B}
4	C	[D, G]	{S, A, B, C}
5	D	[G]	{S, A, B, C, D}
6	G ← GOAL	[]	{S, A, B, C, D, G}

BFS path found: S → A → C → G (path cost using edge weights = 1+2+6 = 9). Nodes removed from frontier: 6.

(b) DFS from S

Stack initially [S]; push children so alphabetically first child is popped first (push in reverse alpha order). Stack written top-on-left.

Iter	Node Popped	Stack after Pushing Children
1	S	[A, B] (push B then A so A is top)
2	A	[C, D, B] (A's unexplored children C,D; push D then C)
3	C	[G, D, B] (C's unexplored child G)
4	G ← GOAL	[D, B]

DFS path found: S → A → C → G (path cost = 9). Nodes removed from frontier: 4.

(c) Uniform-Cost Search from S

Priority queue ordered by g(n). Entries written Node:g. Replace entry when a cheaper path to a frontier node is found. Goal test on expansion (removal from frontier).

Iter	Node Expanded	g(n)	Priority Queue after Expansion
1	S	0	[A:1, B:4]
2	A	1	[C:3, B:4, D:6]
3	C	3	[B:4, D:6, G:9]
4	B	4	[D:5, G:9] (B→D cost 1 → g=5 < 6, replace D:6)
5	D	5	[G:7] (D→G cost 2 → g=7 < 9, replace G:9)
6	G ← GOAL	7	[]

UCS path found: S → B → D → G (path cost = 4+1+2 = 7). Nodes removed from frontier: 6.

(d) Comparison table

Strategy	Path Found	Path Cost	Nodes Removed	Optimal Here?
BFS	S → A → C → G	9	6	No
DFS	S → A → C → G	9	4	No
UCS	S → B → D → G	7	6	Yes

(e) Interpretation

• Why UCS removed as many nodes as BFS yet found a cheaper path: Both expand nodes in an order that eventually reaches G after six removals. BFS expands by depth (unweighted), so it finds the shallowest path S-A-C-G (cost 9). UCS expands by path cost, discovers the cheaper route via B and D (cost 7), and correctly replaces the more expensive G:9 entry with G:7.

• Why DFS needed the fewest nodes but cannot be trusted: In this particular graph the left-most branch leads quickly to G, so only four nodes are removed. In general DFS can dive into an infinite branch or a very long path and is neither complete nor optimal in infinite or cyclic spaces without additional safeguards.

• Why an explored set is needed (BFS iteration 3 example): When B is dequeued its neighbour D is already in the queue (added when A was expanded). Without an explored/frontier check the algorithm would re-insert D, causing redundant work and, in graphs with cycles, possible non-termination. The explored set guarantees each node is expanded at most once.

Question 4: Search with Partial Information – Belief States

Topic: Problem formulation, search with partial information (sensorless and percept-based belief-state search)

Physical states (Agent, Left, Right):

State	Description	State	Description	State	Description
S1	(L, D, D)	S5	(L, C, D)	S7	(L, C, C)
S2	(R, D, D)	S6	(R, C, D)	S8	(R, C, C)
S3	(L, D, C)	S4	(R, D, C)		

(a) Sensorless problem formulation

• Initial belief state: b_0 = {S1, S2, S3, S4, S5, S6, S7, S8} (all 8 physical states possible)

• Actions: Left, Right, Suck (same deterministic effects on every physical state)

• Goal test: a belief state b is a goal iff every physical state in b has both squares Clean, i.e. b ⊆ {S7, S8}

• Path cost: 1 per action (unit cost)

(b) Belief-state evolution under [Right, Suck, Left, Suck]

Step	Action	Belief State after Action
0	(start)	{S1, S2, S3, S4, S5, S6, S7, S8}
1	Right	{S2, S4, S6, S8} (all states now have agent on Right)
2	Suck	{S4, S8} (Right square cleaned → only (R,D,C) and (R,C,C) remain)
3	Left	{S3, S7} (agent moves to Left; dirt status unchanged)
4	Suck	{S7} (Left square cleaned → only (L,C,C) remains – GOAL)

(c) Percept-based belief-state update

Local sensor reports [location, status of current square]. Prediction = apply action to every state in current belief; Update = retain only states consistent with the percept.

Step	Action	After Action (Predicted)	Percept	After Percept (Updated)
0	(start)	{S1..S8}	[Left, Dirty]	{S1, S3} (only L & Left=Dirty)
1	Suck	{S5, S7} (Left cleaned)	[Left, Clean]	{S5, S7} (both consistent)
2	Right	{S6, S8} (agent → Right)	[Right, Dirty]	{S6} (only Right=Dirty)

(d) Number of belief states

With 8 physical states the power set contains $2^8 = 256$ possible belief states (including the empty set, which is unreachable). Searching over belief states is harder because the state space is exponentially larger (2^n versus n). Each belief-state transition may also be more expensive to compute.

(e) Interpretation

• Why percepts shrink the belief state: Each percept is a piece of evidence that eliminates physical states inconsistent with the observation. After [Left, Dirty] only two of the original eight states remain; subsequent percepts continue to prune the set until a singleton (or a goal set) is obtained.

• Difference from the sensorless case: In the sensorless problem the belief state changes only by the deterministic effect of actions; no information is gained, so the agent must find a plan that works from every possible physical state simultaneously. With sensors the agent interleaves actions and observations, continually reducing uncertainty.

• Why the sensorless plan is still rational: A rational agent maximises expected performance given the information it has. When no sensors are available the only information is the initial belief (the whole state space). The open-loop sequence [Right, Suck, Left, Suck] guarantees that both squares become clean from every possible start state; therefore it maximises expected performance under total uncertainty and is rational for a sensorless agent.